EVOKE: A KV-Cache Memory Hierarchy with Recompute-Free Block Recovery for LLM Serving

Anish Shrestha
Independent Researcher
siranishshrestha@gmail.com
github.com/Anyesh/EVOKE

Abstract

Production LLM serving treats cache overflow as terminal: truncate the oldest history or re-prefill on every call. EVOKE argues eviction should be a reversible cut, managed like virtual memory: a "page fault" on an evicted block restores the exact K and V the model first computed, never a re-encoded substitute. Cold blocks leave via cheap position-metadata updates, and any block can be spliced back at a new logical position with one RoPE phase shift, with no forward pass on either path. A retrieval-shaped layer chooses *which* block; the substitution is identity-keyed. We implement this as C primitives in a forked llama.cpp plus a Python policy layer and an OpenAI-compatible server, evaluated across four model families on a single RTX 4070 Ti SUPER (16 GB VRAM). The recovery primitive runs 5.9–7.5$\times$ faster than re-prefilling the same tokens over the full save+load lifecycle on Qwen 2.5 7B (2.6–2.8$\times$ on Llama 3.1 8B; 20–32$\times$ on raw load). Across three benchmarks, every recovery-less baseline (recency, StreamingLLM, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), EVOKE-discard/breadcrumb) clusters at 0–43% multi-fact pass rate while every recovery-bearing policy reaches 48–100%, reproducing at b=1024 on hybrid Mamba/Attention Qwen 3.5 9B (68%, n=5) and MoE Qwen 3.6 35B-A3B (52%). The eviction-scoring choice within the cluster is regime-dependent (InfLLM (Xiao et al., 2024b) at K=8 beats EVOKE at tight budget with non-overlapping CIs, Section 7.3); the load-bearing contribution is the recovery primitive both depend on.

1 Introduction

Production LLM serving handles cache overflow in one of two ways. The server either truncates the oldest history in the spirit of StreamingLLM, or it re-prefills the full conversation at every API call, which is what the OpenAI chat-completions default plus prefix caching amounts to. Truncation discards information that may be needed later. Re-prefill is expensive and pays the cost on every turn regardless of whether the prior context turned out to matter. Neither approach has any model of what content the agent will actually need next.

This paper argues that the right abstraction is the KV cache as a memory hierarchy with a recompute-free recovery primitive (Figure 1). Hot tokens stay in the active cache. Cold blocks are evicted to host RAM at metadata cost. When a future turn needs an evicted block, the block is spliced back into the active cache via a direct K and V tensor write, RoPE-rotated to its new logical position, with no forward pass. Prefix caching only avoids re-decoding an unchanged prefix and does nothing once the cache exceeds budget; Section 4 draws the precise line between EVOKE's identity-keyed recovery and similarity-retrieved re-encoding (RAG).

The contributions of this paper are three:

Preprint. Under review.

1. **Three new C primitives in a forked llama.cpp.** `llama_kv_block_save` and `llama_kv_block_load` serialize a position range's K/V tensors to a host buffer and splice them back into the unified cache with per-cell RoPE re-anchoring (Section 3). `llama_attn_capture_*` taps per-head softmax attention weights from one or more transformer layers into a host buffer once per decode (Section 3.3). Together they enable recompute-free identity-keyed K/V recovery in the unified attention cache and attention-aware eviction scoring; the fork is also extended to support hybrid Mamba+Attention memory and mrope position shifts, so the same primitives work across pure attention (Qwen 2.5), hybrid Mamba (Qwen 3.5), and MoE-with-thinking (Qwen 3.6 35B-A3B) (Section 6).
2. **A Python policy layer and OpenAI-compatible server** that turn the primitives into a usable system. The eviction scorer combines retrieval-tuned embedding coherence (`bge-small-en-v1.5`), the model's own attention mass, recency, and harness-supplied priority. Recovery is routed through three pluggable backends (discard, breadcrumb, kv_restore). The 2^3 smart-recovery factorial in Appendix A.4 attributes the bulk of the gain to a single policy decision (K1: scoring blocks against the raw user-message text through a dedicated retrieval-tuned embedder rather than pooling LM hidden states, marginal +72 pp); running the recovery splice before the new user-message tail is decoded (K2) contributes a conditional +20 pp only when K1 is on, and the resident-gate (K3) has no measurable effect on the benchmark we ran the factorial against (Section 4, Section 5, Appendix A.4). The server reuses prefix-matched KV across turns and pools per-session KV state so a single model in VRAM can host many concurrent agent sessions.
3. **A head-to-head evaluation against seven baselines across three model families, with the eviction-scoring winner regime-dependent.** On the needle-in-a-haystack benchmark at $\sim$3.6$\times$ compression, EVOKE recovers 76 – 100% of needles across the two end-to-end architectures we ran (96 – 100% on Qwen 2.5 7B, 76 – 88% on Llama 3.1 8B; 25 (needle, depth) cells per architecture), while every recovery-less baseline (recency, StreamingLLM, EVOKE's own discard and breadcrumb backends, H2O (Zhang et al., 2023), and SnapKV (Li et al., 2024), all reimplemented in the same harness) flattens at 0 – 44% at the tightest budget, with SnapKV climbing to 68 – 84% on Llama NIAH at looser budgets as a documented single-needle exception (Appendix A.9). On the harder multi-fact n=15 sweep, a same-substrate InfLLM (Xiao et al., 2024b) adaptation at K=8 *statistically separates* from EVOKE at the tightest budget on both architectures (Llama b=512: InfLLM 81.3% [71.1, 88.5] vs evoke_attention 60.0% [48.7, 70.3], non-overlapping Wilson CIs; Qwen b=512: InfLLM 81.3% [71.1, 88.5] vs evoke_kv_restore 50.7% [39.6, 61.7], also non-overlapping). EVOKE pulls ahead only at the loosest budget. Both policies use the same recompute-free recovery primitive; the win at tight budget is the larger-K pure-retrieval selection over EVOKE's K=4 attention-driven scorer. The load-bearing contribution is therefore the primitive that both depend on, with the attention scorer as a regime-targeted improvement on top of it. On a 14-turn planted-fact agentic head-to-head (n=15, budget 1024), EVOKE matches `no_eviction`'s recall at $\sim$3.6$\times$ smaller cache footprint and at `truncate`-parity wall-clock (`truncate` also passes here because the planted fact stays in the natural recency window). When the fact is buried past recency in the agent_bench scaffold (Section 7.2), every recovery-less strategy fails the probe at every budget. Across block sizes from 20 to 1280 tokens, the `kv_block_load` primitive runs in 0.5–16 ms across the two architectures we measured (0.5–7 ms on Qwen 2.5 7B, 1.78–15.66 ms on Llama 3.1 8B) while the equivalent re-prefill takes 12–237 ms (20–32$\times$ speedup on Qwen, 7–15$\times$ on Llama, gap widens linearly with block size; 5.9–7.5$\times$ on Qwen and 2.6–2.8$\times$ on Llama over the full save+load lifecycle) (Section 7).

2 Related Work

Eviction-only KV compression. StreamingLLM (Xiao et al., 2024a) keeps a small set of attention-sink tokens at the cache head plus a sliding recent window and drops everything in between, working well for recency-only tasks but failing on earlier recall. **H2O** (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) score KV entries by accumulated attention weight and drop the lowest-scoring; **SnapKV** (Li et al., 2024) and **Ada-KV** (Feng et al., 2024) subsequently critiqued the H2O family for biasing against recent tokens (cumulative scoring under-counts tail positions)

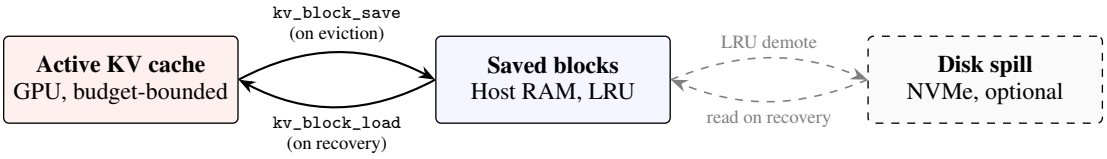

Figure 1: EVOKE's memory hierarchy. The active KV cache holds the hot working set on GPU within a configured budget. On eviction, a block's K and V tensors are serialized to host RAM via `kv_block_save`; recovery reverses the operation with `kv_block_load`, splicing the original tensors back into the active cache at a new logical position with one RoPE phase shift and no forward pass. An optional disk spill tier extends host RAM at NVMe latency.

and proposed single-pass selection and head-aware budgets. All treat eviction as one-way: dropped K/V is gone. EVOKE adopts the same attention-weight signal in its multi-signal scorer (Section 4), keeps the sink-protection idea (sink-tagged blocks score 1.0 unconditionally), combines attention with recency as separate weighted signals to sidestep the recent-bias issue, and pairs eviction with a recovery path so an eviction that turns out to be wrong is recoverable rather than terminal. H2O and SnapKV are reimplemented as same-substrate baselines atop EVOKE's `AttentionScorer` infrastructure (H2O: cumulative attention with no decay, 10% recent-tail guard; SnapKV: one-shot 32-token observation-window snapshot at the end of every user-message decode) and benchmarked head-to-head in Sections 7.2 to 7.3.

Retrieval and external-memory families. RAG (Lewis et al., 2020) stores text in a vector database and re-encodes retrieved passages on every call; the model has no continuity with how those tokens were originally attended. EVOKE keeps the original K and V tensors the model computed at first sight of the tokens and splices them back at their original positions, with Section 4 drawing the precise line between identity-addressed recovery (which may still use similarity to choose *which* blocks to bring back) and similarity-retrieved re-encoding. **InfLLM** (Xiao et al., 2024b) maintains a block-level external KV memory and selects blocks back into the attention window via similarity, holding retrieved blocks in a separate memory segment the attention layer reads alongside the active window. EVOKE shares the block-level memory-hierarchy framing but splices recovered blocks back into the unified contiguous KV cache (RoPE re-anchored to the new logical position), and benchmarks InfLLM as a same-substrate row in Sections 7.2 to 7.3: aggressive eviction to a sinks-plus-local-window footprint with per-turn top-$K{=}8$ smart recovery splicing the externally-held blocks back into the unified cache. **Compressive transformers** (Rae et al., 2019) and related infinite-memory architectures compress old KV into a smaller summary; EVOKE does not compress, trading host RAM proportional to evicted content for exact recovery.

Cache infrastructure and mechanics. vLLM / PagedAttention (Kwon et al., 2023) splits the KV cache into fixed-size physical blocks indexed by a virtual page table; **SGLang's RadixAttention** (Zheng et al., 2024) extends this with a tree of cached prefixes shared across requests. **LMCache** (LMCache Team, 2024) stores prefix-matched KV bytes in a layered cache shareable across vLLM instances, and **DistAttention** (Lin et al., 2024) disaggregates attention computation across a cluster. These cache infrastructures and EVOKE's policy layer address orthogonal concerns: paged virtual addressing handles how cached bytes move and are shared across requests, while EVOKE handles which blocks to evict and how to splice them back. EVOKE's recovery primitive is a typed read/write of K and V keyed by (layer, head, position range), so its algorithmic content is independent of whether the underlying KV layout is contiguous (as in our llama.cpp prototype) or virtually paged. Porting the C primitives onto a page-table-resolved gather/scatter is engineering work we name explicitly as future work (Section 9); this paper presents the primitive and policy on a flat unified cache as the simplest substrate that exposes both K and V tensors at addressable offsets. **RoPE re-anchoring** is enabled by RoFormer (Su et al., 2024): shifting a K row from one position to another reduces to a single rotation, machinery llama.cpp already exposes for its own `seq_add` shift, which EVOKE reuses on the recovery path. **Retrieval embeddings** for smart-recovery scoring use BGE (Xiao et al., 2024c), specifically `bge-small-en-v1.5` via the `fastembed` runtime; the choice of a dedicated retrieval embedder rather than LM hidden states is what makes block-vs-query similarity discriminative on retrieval-style workloads (Section 7.2).

3 The C Primitives

The goal is to save a range $[p_0, p_1)$ of cells from sequence `seq_id` into a host buffer, then restore that buffer into the cache at a new starting position `new_p0`. Restoration must produce the same K and V tensors the model would attend to if positions `new_p0` through `new_p0` $+ (p_1 - p_0)$ had been decoded in place.

The primitives are implemented in `src/llama-context.cpp` and exposed in `include/llama.h`:

```
size_t llama_kv_block_save(llama_context * ctx, llama_seq_id seq_id, llama_pos p0,
                           llama_pos p1, uint8_t * dst, size_t cap);
bool   llama_kv_block_load(llama_context * ctx, llama_seq_id seq_id,
                           const uint8_t * src, size_t size, llama_pos new_p0);
```

`llama_kv_block_save` adapts the existing `state_write_meta` and `state_write_data` paths but filters to cells whose `pos` is in $[p_0, p_1)$ for `seq_id`. Per layer, we read the per-cell K row and V row via `ggml_backend_tensor_get`. We also store each cell's original `pos` in the buffer because the deferred RoPE shift will need it.

`llama_kv_block_load` adapts the `dest_seq_id != -1` path of `state_read_data` with one change: instead of `seq_rm(seq, -1, -1)`, which would wipe the whole sequence, we clear only `seq_rm(seq, new_p0, new_p0 + n_cells)`. We build a ubatch over $[$`new_p0`$,$ `new_p0` $+$ `n_cells`$)$, call `find_slot`, `apply_ubatch`, and write the saved K and V via `ggml_backend_tensor_set`. We re-anchor RoPE by setting `shift`$[i]$ $=$ `new_pos` $-$ `original_pos` per cell, then run the existing K-shift graph via `memory_update`. V is never rotated.

This path runs no model forward pass: `llama_decode` is never called, the per-block cost is $O($`n_cells` $\times$ `n_layers` $\times$ `n_embd_kv` $\times$ `sizeof(dtype)`$)$ of host-to-device memcpy plus a single K-rotation graph (the deferred RoPE shift llama.cpp already runs for its own `seq_add` path), and the V tensor is never touched after the initial save. Throughout this paper, "recompute-free" is shorthand for "no model forward pass," not literal zero compute; the latency numbers in Section 7.1 measure end-to-end load including the rotation.

3.1 Flash Attention is non-optional

With Flash Attention (Dao et al., 2022; Dao, 2023) disabled, V is stored transposed (`v_trans=true`). The `state_read_data` and `state_write_data` V loop then iterates `n_embd_v_gqa` times per layer. For Qwen 2.5 7B that is 28 layers times 512 `n_embd_v_gqa` per layer, which works out to 14,336 synchronous CUDA launches per load. With Flash Attention enabled (`v_trans=false`), V is row-aligned like K and the fast path runs 56 launches per load (two per layer, one for K and one for V). Measured impact on the eval host for a 20-token block: `kv_block_load` dropped from 133 milliseconds with FA disabled to 0.48 milliseconds with FA enabled. Flash Attention is therefore a performance precondition for the contribution: the splice is functionally correct with FA off, but the $\sim$280$\times$ slowdown collapses the speedup over re-prefill that the recovery primitive exists to deliver.

Deployment-substrate consequence. The Flash Attention requirement excludes the substrates on which FA is not available or not enabled by default: CPU-only inference, the older Vulkan and Metal backends that do not yet ship the fused softmax path, and pre-Ampere CUDA hardware. EVOKE's usable surface is therefore Ampere-or-later CUDA with FA on. This is the same surface modern paged-attention serving stacks (vLLM, SGLang) target by default, so the requirement is consistent with production deployment assumptions, but it is a real exclusion that future-work substrate ports must reckon with rather than a free assumption.

3.2 Cross-architecture extensions

The fork's helper `llama_get_kv_cache`, which resolves a context's `llama_memory_t` to an attention KV cache, is extended to unwrap two more memory types: `llama_memory_hybrid` (attention plus recurrent), via `get_mem_attn()`, and `llama_memory_hybrid_iswa` (attention with interleaved sliding-window attention plus recurrent), via `get_mem_attn()->get_base()`. Hybrid models carry a fixed-size recurrent state per layer that does not need eviction, so EVOKE operates on

the attention component alone. The recurrent contribution to memory is preserved naturally as the model carries its blurry state forward.

For mrope models (Qwen 3.5 and later, anything with `n_pos_per_embd > 1`), the upstream `seq_add()` and `get_can_shift()` returned false outright. We removed those guards. `build_rope_shift` already has an mrope workaround that treats the temporal shift as a NeoX-style whole-vector rotation. Cells store a single temporal `pos` plus a separate `ext` payload for spatial axes, and shifting only the temporal pos is semantically correct for our position-only re-anchoring.

For hybrid Mamba plus Attention memories, `llama_memory_hybrid::seq_rm` dispatches to the recurrent half first. The recurrent half refuses partial tail rollback (a Mamba state cannot be sliced) and returns false, and the hybrid wrapper then returns false without mutating either sub-cache. A naive caller that interprets the return as success and updates its position bookkeeping will corrupt the cache on the next `llama_decode`; EVOKE's Python `evict_ranges` now reads the return value and leaves bookkeeping intact on rejection.

For cases where the policy wants to drop attention cells without touching the recurrent state, two attention-only primitives, `llama_kv_block_seq_rm` and `llama_kv_block_seq_add`, are exposed in the fork. They reach the attention sub-cache directly via `llama_get_kv_cache` and bypass the hybrid wrapper, supporting a future no-shift eviction mode that the current policy layer does not yet exercise.

For iSWA models (Gemma 3 and 4, which interleave global and sliding-window attention across layers), the fork's `llama_kv_block_save` and `llama_kv_block_load` save and restore both the base and SWA sub-caches via a newly added `llama_get_kv_cache_swa()` helper. The buffer layout for iSWA models is `[base block bytes][4-byte swa size][swa block bytes]`. For non-iSWA models the layout is unchanged. Python-side wiring to load a Gemma model and verify end-to-end recovery through EVOKE's stack is not yet in place; only the fork primitives have been verified.

3.3 Attention-weight capture

The multi-signal scorer (Section 4) needs to know which past tokens the model is actively attending to as the conversation progresses. The strongest possible signal for this is the model's own attention, the bilinear form $\texttt{softmax}(QK^\top/\sqrt{d})$ that determines the forward pass itself.

Flash Attention fuses the softmax inside its CUDA kernel and never materializes the attention weights as a tensor, and EVOKE requires Flash Attention to be on (Section 3.1: it dictates V's row-aligned layout, which is what makes the recovery primitives fast). Reading `kq_soft_max` from the existing graph is therefore not an option, and modifying the Flash Attention kernel itself to optionally emit weights would mean a correctness-sensitive change across the CPU, CUDA, Metal, and Vulkan backends.

The lower-risk path is to compute a second softmax in parallel for one or more chosen layers. After Q and K are computed and permuted (the same Q and K that Flash Attention consumes), we add a separate graph node that computes $\texttt{softmax}(QK^\top, \texttt{scale} = \texttt{kq\_scale}, \texttt{mask} = \texttt{kq\_mask}, \texttt{sinks} = \texttt{sinks})$. The output is named `evoke_attn_capture` and force-expanded via `ggml_build_forward_expand` so the scheduler does not drop it as dead code. When `llama_decode` returns, `llama_context` copies the tensor data into a caller-owned host buffer via `ggml_backend_tensor_get`. The main attention path is unchanged: Flash Attention still runs, and the model output is byte-identical with capture off versus capture on.

The C API surface (`llama_attn_capture_set_layer`, `_set_layers`, `_set_buffer`, `_get_dims`, `_get_written`, declared in `include/llama.h`) is given verbatim in Appendix C.4. `set_layer` captures a single layer; `set_layers` writes up to 16 layer slabs per decode into a host-resident float32 buffer at layout `[n_layers, n_query, n_heads, n_kv]`. For Qwen 2.5 7B at decode (`n_query`=1, `n_heads`=28, `n_kv` up to 16K), one layer slab is roughly 1.8 MB per step; four layers fit in 7.2 MB.

Validation. A real-model smoke test (`scripts/verify_attn_capture.py`) decodes a 31-token prompt on Qwen 2.5 7B with layer 20 tapped. It asserts that per-query softmax sums to 1.0, that values lie in $[0, 1]$, that the causal mask is respected (weights above the diagonal are exactly zero),

and that the number of legal nonzero cells matches $\sum_i (i+1) \cdot n_{\text{heads}}$ exactly. A second test (`scripts/verify_attn_capture_multi.py`) captures three layers at once (layers 4, 14, 20) in a single 29-token decode and asserts the same invariants per layer plus that distinct layers produce distinct attention patterns. Both pass on the eval host. The side-compute is numerically close but not bit-identical to the Flash Attention softmax, since the two paths run in different graph nodes; we verified this against non-FA mode for one test prompt and confirmed the difference is below the rounding tolerance of the f32 buffer.

For the scorer experiments in Appendices A.1 and A.6, we use a single mid-layer (layer 20 for Qwen 2.5 7B), which we found sufficient to drive the relevance signal. Multi-layer aggregation is exposed in the API for future scorer designs.

3.4 Why eviction does not corrupt the cache

Eviction is two engine calls. `seq_rm(seq, p0, p1)` marks the cells in $[p_0, p_1)$ free in the unified KV buffer, releasing their K and V rows. No dangling reference survives because Q is never cached: it is recomputed each decode step and discarded. `seq_add(seq, p1, end, delta)` with $\delta = -(p_1 - p_0)$ then updates the position metadata of the survivors to close the gap and queues a deferred RoPE shift by δ on their K rows. K and V bytes are not moved in memory; only the per-cell positions change.

llama.cpp applies the queued shift lazily at the next attention compute. Because RoPE expresses position as a multiplicative phase on K (and Q), with $K(\text{pos}) \cdot Q(\text{pos}_q)$ collapsing to a function of $(\text{pos} - \text{pos}_q)$ (Su et al., 2024), re-anchoring a K row from old position p to new position $p + \delta$ is exactly one rotation $R(\delta \cdot \theta_i)$ per dimension pair. V is positional-free and untouched. After the shift, $Q_{\text{new}} \cdot K_{\text{survivor}}$ returns the same relative-position dot product the model would compute if the evicted range had never been decoded. The mrope case (Qwen 3.5+) follows the same logic via the temporal axis (Section 3.2); the hybrid case forwards `seq_rm`'s rejection of partial recurrent rollback to the caller without mutating the cache. Eviction is therefore a topological cut, not partial erasure: failure modes are information loss (the model no longer has K and V for an evicted fact and will hallucinate or refuse) rather than corruption, which is the failure mode `kv_restore` addresses by splicing the saved K and V back at a fresh contiguous position with one further RoPE shift.

4 The EVOKE Policy Layer

The policy layer lives in `evoke/manager.py`, `evoke/scorer.py`, and `evoke/attention_scorer.py`. Each active block carries a score in $[0, 1]$ used by the watermark policy: when `active_tokens` crosses `high_watermark` $\cdot$ `budget`, the manager evicts the lowest-scoring blocks down to `low_watermark` $\cdot$ `budget`.

What is actually load-bearing. We describe the full multi-signal scorer below for completeness, but the 2^3 factorial in Appendix A.4 attributes the bulk of the smart-recovery gain on NIAH at b=512 to a single policy decision: scoring blocks against the raw user-message text through a dedicated retrieval-tuned embedder rather than pooling LM hidden states (K1, marginal +72 pp). Running the recovery splice before the new user-message tail is decoded (K2) contributes a conditional +20 pp only when K1 is on. The third decision (K3, the resident-gate) has no measurable effect on the benchmark we ran the factorial against. The multi-signal weighted-linear-combination scorer below is therefore best read as the framework within which the load-bearing primitive (the recovery splice in Section 3) and the load-bearing policy decision (the retrieval embedder) compose, not as the contribution. The attention-mass signal is an additional regime-targeted improvement that pays off when the model's attention concentrates on a single recoverable target (Appendix A.1); on the multi-fact regime its value is budget-dependent and a larger-K pure-retrieval recovery can outperform it (Section 7.3).

The score is a weighted linear combination of up to five signals, lifted by a source-type floor, multiplied by a harness-supplied priority, and clamped:

$$\textit{raw} = \frac{w_{\textit{attn}} \cdot \textit{attn} \;+\; w_{\textit{rec}} \cdot \textit{recency} \;+\; w_{\textit{coh}} \cdot \textit{coherence} \;+\; w_{\textit{recov}} \cdot \textit{recovery\_strength}}{\sum w_i}$$

$$\textit{score} = \min\bigl(1,\ \max(\textit{raw},\ \textit{source\_floor})\ \cdot\ \textit{block.priority}\bigr)$$

Sink blocks short-circuit to *score* $= 1.0$ unconditionally. The four signals in the numerator are explained below; the fifth (*recovery_strength*) is a model-history term covered separately in the recovery-aware paragraph.

Attention (Section 3.3). Per-block softmax mass landing in that block over the last `n_window` decode steps, EWMA-weighted by `decay`. Defaults are a 64-step window and a decay of 0.95. This is the model's own vote: blocks the recent query tokens actually attended to score high. When attention capture is unavailable (a stock llama.cpp build, or `w_attention=0`), this signal is absent and the score falls back to recency plus coherence.

Task-focus coherence. The scorer maintains a single task focus embedding rather than averaging the last N message embeddings. On each new user message the focus either updates via EMA (when the new message is coherent with the running focus, with cosine similarity to the focus $\geq$ `task_boundary_threshold`, defaulting to 0.3 in the cosine sense, not as a weight), or it snaps to the new message. The snap is detected implicitly via the cosine drop, or signaled explicitly by the harness via `evoke_task_boundary=true`. Snapping makes blocks coherent with a prior task lose their coherence score in one scoring pass instead of decaying over five turns under the old rolling average.

Per-block embeddings can be drawn from two sources, switchable via `EvokeConfig.use_retrieval_embeddings`. The default mode pools the model's own intermediate hidden states across the block tokens (`block_embedding_strategy="mean"`) — cheap because the LM has already computed them, but the LM hidden state carries a strong common-mode component that collapses cosine similarities into a narrow $0.85 - 0.93$ band across any two paragraphs in the same corpus, leaving no headroom for fine-grained retrieval. The opt-in retrieval mode embeds each block's decoded text through a dedicated retrieval-tuned model (`BAAI/bge-small-en-v1.5` via `fastembed`, $\sim$30 MB, $\sim$10 ms per text on CPU). The cosine band widens to $0.4 - 0.9$ on the same workload, and the smart-recovery policy below (Section 5) can actually distinguish a needle block from haystack noise. Both block embeddings and the probe query embedding must come from the same space; the policy layer enforces this by routing both through the same embedder.

Recency. Exponential in the per-block distance from the current write position. A stability prior that prevents thrashing on a single high-attention spike.

Sink protection and source-type floors. Blocks marked `is_sink=True` (the first `sink_count` positions) score 1.0 unconditionally, which is the StreamingLLM-style attention anchor. USER and ASSISTANT turns get a score floor (defaults of 0.6 and 0.5 respectively) so that the conversation backbone is not evicted before document content.

Recovery-aware eviction. A fifth, model-history-driven signal records whether the model recently signaled a block matters by recovering it. Each `ActiveBlock` carries a `recovery_strength` float defaulting to 0.0. `EvokeManager.recover()` sets it to `recovery_strength_init` (default 1.0) on a fresh recovery; `EvokeManager.tick_turn()` multiplies every active block's strength by `recovery_decay` (default 0.7) at the start of each user turn. The scorer adds $w_{\text{recovery}} \cdot$ `recovery_strength` to the weighted sum. Default `w_recovery` $= 0$ preserves the prior four-signal behavior; setting it positive closes the recover-then-immediately-evict thrash that the session-length sweep (Appendix A.7) exposed at $T{=}28{+}$: a freshly-recovered block survives the next eviction pass even when its recency and coherence scores would have evicted it again, then decays back to ordinary eviction eligibility over $\sim$5 turns. The mechanism is structural, not a heuristic guardrail: eviction now respects the model's own prior relevance signal rather than evicting on recency and coherence alone.

EVOKE ships two named scorer recipes against the equation above. The `EvokeConfig` default (`w_attention=0.0`, `w_recency=0.4`, `w_coherence=0.6`) is the fork-independent fallback: attention scoring is off out of the box, and a stock llama.cpp build that lacks the attention-capture primitive runs the recency-plus-coherence heuristic without modification. The recommended config (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`) opts in to the multi-signal scorer when the EVOKE fork build is available; this is the config we recommend for agent workloads with fork access. Across every head-to-head bench in Section 7, the two recipes perform statistically identically:

NIAH on Qwen 2.5 7B is 96/100/100% for both at budgets 512/1024/2048 (Section 7.2), NIAH on Llama 3.1 8B is 88% across all capture-layer choices for both (Appendix A.9), and multi-fact n=15 pass-rate CIs overlap at every budget (Section 7.3). We recommend the multi-signal recipe not because it wins, but because attention is a more principled relevance signal than recency or coherence: it is the model's own per-block attention pattern rather than a heuristic prior. The fork-independent default exists for substrate-constrained deployments. Note that the `w_coherence` weight (0.3 or 0.6 depending on recipe) and `task_boundary_threshold` (0.3 cosine) are unrelated despite the shared number; weights are linear combination coefficients while the threshold is a cosine cutoff for topic-shift detection.

Harness-supplied tags. The OpenAI chat-completions request gains three EVOKE-specific fields. `evoke_priority` is a float at least zero that multiplies the block's final score. `evoke_pinned` is a boolean that excludes the block from eviction candidates entirely. `evoke_task_boundary` is a boolean that forces the coherence focus to snap. A harness like opencode or Claude Code that knows which file reads are central to the current task can set `priority=2.0` to keep them resident, and one-shot tool output can be marked `priority=0.3` so it is preferentially dropped under budget pressure. The fields default to `1.0`, `false`, and `false`, so harnesses that ignore them get the model-and-heuristic behavior.

Pinned blocks are excluded from `_evictable_blocks` alongside sinks and the current-turn pin. `pin_generated=True` (the default) protects blocks created during the currently decoding turn so that generation does not immediately evict itself.

Recovery backends (`evoke/recovery.py`). Three pluggable strategies share a `RecoveryBackend` protocol.

- `DiscardBackend`: evicted content is gone. The agent must re-derive it.
- `BreadcrumbBackend`: a compact marker (`key`, token count, eviction step) is retained per evicted block. The agent (or harness) can choose to re-fetch the original text and call `add_context` to re-decode it.
- `KVRestoreBackend`: per-block K and V are saved to host RAM via `kv_block_save`. `recover(key)` looks up the saved buffer, calls `kv_block_load` at the next available position, and re-anchors RoPE. When a configurable RAM budget would otherwise drop an LRU victim, the backend can optionally spill the K and V bytes to disk under `kv_restore_spill_path`. Recovery then reads back from disk before splicing, at the cost of one NVMe read.

Block identity and where EVOKE sits relative to RAG. EVOKE is a hybrid by construction: a retrieval-shaped selection layer chooses which evicted block to bring back, and an identity-keyed substitution layer splices that exact block's original K and V tensors into the cache. Selection looks like retrieval: smart top-K scoring in Section 5 embeds blocks and queries with `bge-small` and picks by cosine similarity. Substitution does not: every block carries a stable `key` such as `"file:config.py#0"`, `"user#42"`, or `"assistant#43"`, and the recovery call names that key and pastes back the exact bytes the model computed at first decode, never a re-encoding of nearby text and never a similarity-retrieved alternative. The new systems content of this paper lives at the substitution layer (the C primitives in Section 3); the selection layer is acknowledged retrieval and intentionally so.

5 The OpenAI-Compatible Server

The chat-completions API is stateless: every request resends the full message history, and the canonical server response is to re-prefill from scratch (or, with prefix caching, to reuse the unchanged prefix). EVOKE's server (`evoke/server.py`, `evoke/session.py`) does both prefix caching and the eviction-and-recovery cycle under it.

Templating and prefix match. A persistent `Session` with one `EvokeManager` underneath outlives every request. On each incoming request the server templates the messages into raw text using the model's own Jinja template parsed from the GGUF metadata: `llama_chat_apply_template`

handles plain chat, and a Python jinja2 path handles tool-using requests (the C API does not accept a tools array). The templated text is tokenized, then prefix-matched against the session's `cached_tokens` to find the longest token prefix already in the physical KV cache.

Decode and eviction. Tokens past the prefix-match point are routed through `manager.add_context_tokens`, which decodes them, creates blocks, and invokes `_enforce_budget`. Eviction fires here under watermark pressure (Section 4).

Smart recovery. The session runs smart top-K recovery on each user turn (default $k = 4$). A 2^3 factorial over three smart-recovery policy choices (Appendix A.4) attributes the bulk of the NIAH improvement to one of them: using a dedicated retrieval-tuned embedder against the raw user-message text instead of an LM-hidden-state average. Its main effect at NIAH b=512 on Qwen 2.5 7B is +72 pp (14 % $\rightarrow$ 86 % marginal pass rate), because the retrieval path widens the cosine band enough to separate the needle from haystack noise. Whether recovery runs *before* the new user-message tail is decoded interacts with the embedding choice: with the retrieval embedder it adds +20 pp (76 % $\rightarrow$ 96 %) because recovered blocks land as earlier context the model attends back to from the freshest probe, but with the LM-hidden-state embedder it actively hurts by -28 pp because poor retrieval lands wrong blocks in the freshest attention slot. A resident-gate that excludes the current-turn user message from the threshold comparison contributes no measurable effect at b=512 in the factorial; it was put in to prevent a specific depth-95 % failure mode (Section 7.2) and its value, if any, shows up at depths the factorial does not sweep. Per-turn recovery cost is capped at k `kv_block_load` calls.

Generate, strip, stream. Tokens stream out via `engine.generate_next`. The session detects `<think>...</think>` traces and, by default, strips them from the response and evicts the thinking range from the physical cache so that the cached state aligns with the next request's templated prompt. Tool calls are parsed from `<tool_call>` XML and returned in OpenAI's `tool_calls` shape. The SSE stream is token-level and aware of thinking and tool-call markers: the loop runs a small three-state machine (thinking, tool-locked, normal) with a 12-character lookback that prevents partial markers from shipping as content.

Multi-session pool. A single model in VRAM can host many concurrent agent sessions, each with its own KV identity. A `SessionPool` (in `evoke/session.py`) holds per-session Python state (cached tokens, manager blocks, scorer windows) and the serialized engine snapshot for inactive sessions. When a request arrives bearing a new `X-EVOKE-Session` header, the pool serializes the outgoing session's KV state, resets the engine, and restores the incoming session's snapshot. The transition is one memcpy proportional to the active KV size with no recompute. Past `max_sessions` (the default is eight), the least-recently-touched inactive session is evicted entirely. State-swap correctness is verified by `scripts/verify_multi_session.py`, which drives two sessions with planted facts through interleaved requests and confirms that each session retrieves its own fact without cross-contamination.

6 Models Supported

End-to-end verified on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1):

Model	Architecture	Status
Qwen 2.5 7B Instruct	Pure attention, standard RoPE	full bench suite (NIAH, multifact, agent_bench, baseline_bench, concurrency); NIAH 96 – 100% across budgets 512/1024/2048
Llama 3.1 8B Instruct	Pure attention, standard RoPE, 32 transformer layers	full bench suite (Appendix A.9): NIAH 76 / 88 / 88 % across budgets 512/1024/2048; multifact regime crossover reproduces; agent_bench PASS at every budget; `kv_block_load` 1.78–15.66 ms across block sizes 20–1280 tokens (Section 7.1)
Qwen 3.5 9B	Hybrid Mamba/Attention, mrope, thinking	full NIAH grid (25 cells, 84% pass; 4 fail cells are generation-side truncation of the recovered value, not primitive misses) and full multifact grid (25 cells, n=5 seeds, 68 % [48.41, 82.80] vs 0–8% recovery-less, H2O 8%) at budget 1024; thinking-strip selectable via `EVOKE_SUPPRESS_THINKING_STRIP`. Budget sweep (512/2048) not run due to thinking-mode generation cost.
Qwen 3.6 35B-A3B (IQ2_M)	MoE attention, mrope, thinking	full NIAH grid at budget 1024 (64% vs baselines 16%) and full multifact grid (25 cells, n=5, 52 % [33.50, 69.97] vs 0% every baseline including H2O); aggressive IQ2 quant likely accounts for the lower absolute number. Budget sweep not run; 35B at IQ2_M sits near the 16 GB VRAM ceiling at model weights alone.

7 Evaluation

The evaluation is organized around two claims, in order of importance. Section 7.1 measures the recompute-free recovery primitive (Section 3) against the alternative of re-prefilling the same tokens. Sections 7.2 and 7.3 measure the policy layer (Sections 4 and 5) at recall tasks, separating the structural recovery-bearing-vs-recovery-less divide from the narrower head-to-head against InfLLM. Section 7.4 measures wall-clock against a vanilla server and against truncation, where we make no speed claim. Section 7.5 reports the negative substrate-portability result. Detailed ablations (scorer-knob factorial, host-RAM degradation, session-length scaling, sustained-load concurrency, KV-quant comparison) are in Appendix A.

7.1 Primitive latency: recompute-free recovery versus re-prefill

Engine-level profile on the eval host (RTX 4070 Ti SUPER, 16 GB VRAM) with Flash Attention enabled, generated by `scripts/profile_recover.py` across block sizes from 20 to 1280 tokens. `kv_block_load` time is the splice-and-rotate cost. `re-prefill` time is the equivalent `process_tokens` call, the alternative if the same tokens had to be re-encoded. Figure 2 visualizes the result; the table that follows reports both architectures.

	Qwen 2.5 7B Q4_K_M				Llama 3.1 8B Q4_K_M			
n_tok	save	load	re-prefill	speedup	save	load	re-prefill	speedup
20	1.10	0.48	11.90	25×	2.65	1.78	12.58	7.1×
40	1.61	0.70	13.78	20×	3.82	1.94	14.62	7.5×
80	2.76	0.89	19.00	21×	5.82	2.46	19.94	8.1×
160	4.69	1.50	32.60	22×	10.30	3.60	32.55	9.0×
320	8.40	2.41	59.72	25×	19.82	5.77	61.23	10.6×
640	16.37	4.34	118.36	27×	39.05	8.87	121.83	13.7×
1280	31.90	7.25	232.18	32×	74.35	15.66	236.89	15.1×

Table 1: `kv_block_save`, `kv_block_load`, and equivalent `re-prefill` times in milliseconds across block sizes 20 to 1280 tokens. Speedup is re-prefill divided by load (the latency a deployer pays on recovery). The save path is paid once at eviction.

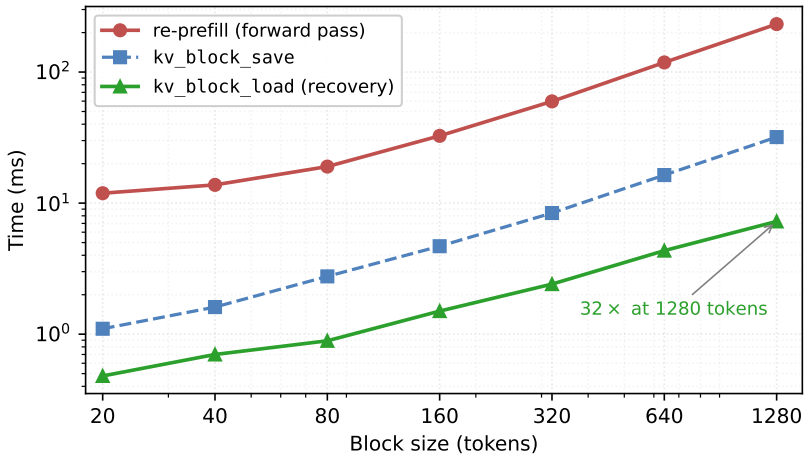

Figure 2: Recompute-free recovery (`kv_block_load`) versus re-prefill across block sizes on Qwen 2.5 7B (RTX 4070 Ti SUPER, Flash Attention enabled). Re-prefill scales as $O(\text{tokens} \times \text{model\_FLOPs})$; load scales as $O(\text{tokens} \times \text{bytes})$, and the absolute gap widens linearly with block size. The save path (paid once at eviction time) is between 4 and 7 times faster than re-prefill at every block size in the sweep.

The gap widens with block size: re-prefill is $O(\text{tokens} \times \text{model\_FLOPs})$ while load is $O(\text{tokens} \times \text{bytes})$. At 1280 tokens host-to-GPU transfer dominates and `kv_block_load` is 32 times faster than re-prefill on Qwen. Re-prefill is essentially identical between architectures (a 7B-class forward pass dominated by model FLOPs); the save and load curves are scaled up by Llama's larger per-token K/V footprint ($\sim$131 KB/token vs Qwen's $\sim$60), dropping the speedup band from 20 to 32$\times$ on Qwen to 7 to 15$\times$ on Llama. Recompute-free recovery remains strictly faster than re-prefill at every block size on both architectures.

Full-lifecycle cost. `save` is paid at eviction time and `load` at recovery time. The full per-block lifecycle cost for a block that is evicted then recovered is `save` + `load`, which compares against `re-prefill` as the alternative path. On Qwen that is 1.58 ms vs 11.90 ms at 20 tokens (7.5$\times$) and 39.15 ms vs 232.18 ms at 1280 tokens (5.9$\times$). On Llama the lifecycle ratio is 2.6 to 2.8$\times$. A block evicted but never recovered pays the save cost without benefit; Section 7.4 measures the full end-to-end cost including save plus the bge-small smart-recovery selection overhead at the session level.

7.2 Recovery is the dividing line, not the eviction scorer

The structural finding across the recall benchmarks is that the presence of a recovery primitive separates passing policies from failing ones, regardless of how smart the eviction selector is. We show this on three benchmarks of increasing difficulty.

Honest framing of the comparison. The recovery-bearing-vs-recovery-less divide is large because recovery-less baselines structurally cannot retrieve evicted content. We report the gap because it quantifies what the recovery primitive enables (rather than asserting the policy is smarter than H2O or SnapKV at their own task of eviction selection). The narrower comparison against the one recovery-bearing baseline (InfLLM at K=8) is in Section 7.3; that is the head-to-head where the eviction scorer is on trial.

Agent-bench: a planted fact past recency. `scripts/agent_bench.py` plants a fact in an early "config file" read (`config.py: maximum retry limit 17 attempts`), fills the working set with ten unrelated file reads, then probes the fact. By the time the probe runs, the config block has almost certainly been evicted under budget pressure. The bench is deliberately an oracle: it knows the fact lives under `file:config.py` and, for the recovery-bearing strategies, recovers exactly that

key before the probe. This isolates the cost of the recovery primitive from the cost of *selecting* which block to recover.

Nine strategies at three KV budgets on Qwen 2.5 7B, `block_size=64`: five EVOKE-substrate policies (recency, streaming_llm, evoke_discard, evoke_breadcrumb, evoke_kv_restore, plus evoke_attention added in Appendix A.1) and three same-substrate baselines (h2o, snapkv, infllm). Table 2 summarizes pass status across all 27 cells; full per-cell numbers (eviction counts, recovery latency) are in Appendix A.1.

strategy	b**=512**	b**=1024**	b**=2048**
recency	fail	fail	fail
streaming_llm	fail	fail	fail
evoke_discard	fail	fail	fail
h2o	fail	fail	fail
snapkv	fail	fail	fail
evoke_breadcrumb	PASS	PASS	PASS
evoke_kv_restore	PASS	PASS	PASS
evoke_attention	PASS	PASS	PASS
infllm	PASS	PASS	PASS

Table 2: Agent-bench probe outcome on Qwen 2.5 7B, `block_size=64`, three KV budgets. Every recovery-less policy fails the probe at every budget; every recovery-bearing policy passes. Llama 3.1 8B reproduces the same divide. EVOKE strategies, breadcrumb, and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail.

The five failing policies produce concrete refusal modes: "The maximum retry limit set in `config.py` is not explicitly mentioned" or "I cannot answer this question without inspecting the `config.py` file." The H2O cumulative-attention selector and SnapKV's question-window snapshot do not change the outcome: their eviction counts are accurate (34, 24, and 4 for H2O/SnapKV; the block holding the fact is gone), but the model has no way to recover the lost context. Even `streaming_llm`, which pins the first four blocks as sinks, fails because the planted fact sits well past the sink prefix. The four recovery-bearing policies all pass at every budget; the cost structure separates cleanly between `breadcrumb` (15 to 17 ms per recovery, re-decode) and `kv_restore` (3 to 4 ms, splice). `evoke_attention` additionally avoids recovery entirely at b=512: the multi-signal scorer assigns the config-file block a high enough score that eviction never picks it in the first place. Appendix A.1 isolates this differential.

Needle-in-a-haystack. `scripts/niah_bench.py` plants a distinctive fact (a needle, e.g., "the secret password for the laboratory vault is icarus-pinwheel-43") at a controlled depth inside a long synthetic haystack of paragraphs about diverse made-up topics. The haystack is deterministic from a fixed seed, with 40 paragraphs at `block_size=64` ($\sim$60 blocks, $\sim$4$\times$ the 1024-token budget). Five distinct needles cover four answer types; five depths cover {5, 25, 50, 75, 95}% of haystack position. Table 3 reports pass rate on Qwen 2.5 7B and Llama 3.1 8B at three budgets, 25 cells per (architecture, policy, budget).

The pattern is clean. Recovery-less policies all pass only at depth 95 % where the needle sits in the recent tail and never gets evicted; at every other depth they cannot recover the lost block. SnapKV on Llama is the one documented exception: its heavy-hitter selection preferentially retains the needle when the needle token has high attention mass during single-needle ingestion, climbing to 44/68/84 % across budgets. It falls back to the recovery-less cluster on the multi-fact bench in Section 7.3, where five sequential probes shift the model's attention across topics and no single block holds top-K mass throughout. The fail cells for evoke variants at b=512 (and InfLLM at b=512 and 1024) are the depth-95 capital cell where smart-recovery still fires on the resident needle and recovered blocks land as freshest context, anchoring the model on post-needle haystack rather than on the naturally-resident needle text; the K and V tensors are correctly spliced (`kv_block_load` runs successfully), but the recovery competes with what was already there.

Multi-fact (n=5 pilot). `scripts/multifact_bench.py` plants five distinct facts in the same session at jittered depths and probes all five in sequence at the end. Each per-fact probe runs its own smart-recovery pass, so the cache churns continuously through five different topic shifts. Five seeds

	Qwen 2.5 7B			Llama 3.1 8B		
policy	b=**512**	b=**1024**	b=**2048**	b=**512**	b=**1024**	b=**2048**
recency	20 %	20 %	40 %	0 %	0 %	0 %
streaming_llm	0 %	20 %	20 %	12 %	20 %	24 %
evoke_discard	0 %	20 %	20 %	12 %	20 %	24 %
evoke_breadcrumb	0 %	20 %	20 %	12 %	20 %	24 %
h2o	20 %	20 %	40 %	20 %	20 %	40 %
snapkv	20 %	20 %	40 %	44 %	68 %	84 %
infllm	**96 %**	**96 %**	80 %	**84 %**	76 %	68 %
evoke_kv_restore	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**
evoke_attention	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**

Table 3: NIAH pass rate, 25 (needle, depth) cells per cell. Recovery-bearing policies (evoke variants, infllm) recover the needle across the full depth range. Recovery-less policies pass only at the recency-naturally-resident depth. SnapKV's heavy-hitter retention on Llama at looser budgets (44/68/84 %) is a documented single-needle exception that does not carry to multi-fact (Table 5).

vary the paraphrase template and the per-fact value. Scoring is hybrid: a string-match-set on multiple value variants is the primary metric, and a local LLM judge (`gemma-4-E4B-it` Q4_K_M, loaded sequentially after the SUT) decides ambiguous answers. Table 4 reports the n=5 pilot on Qwen 2.5 7B at b=1024, where the gap between recovery-bearing and recovery-less policies first appeared. The tighter n=15 extension follows in Section 7.3, where it surfaces the budget-regime crossover against InfLLM.

strategy	**pass rate**	**95 % Wilson CI**
recency	4.0 %	[0.71 %, 19.54 %]
streaming_llm	0.0 %	[0.00 %, 13.32 %]
evoke_discard	0.0 %	[0.00 %, 13.32 %]
evoke_breadcrumb	0.0 %	[0.00 %, 13.32 %]
h2o	0.0 %	[0.00 %, 13.32 %]
snapkv	4.0 %	[0.71 %, 19.54 %]
evoke_attention	48.0 %	[30.03 %, 66.50 %]
evoke_kv_restore	**60.0 %**	**[40.74 %, 76.60 %]**
infllm	**64.0 %**	**[44.52 %, 79.75 %]**

Table 4: Multi-fact n=5 pilot on Qwen 2.5 7B at b=1024, 25 facts per row (5 facts × 5 seeds). Wilson 95 % confidence intervals. Per-seed evoke_kv_restore: {80, 60, 40, 60, 60} %; per-seed infllm: {80, 60, 40, 60, 80} %.

The structural divide reproduces on the cross-architecture sweep: on Qwen 3.5 9B (hybrid Mamba+Attention with thinking) EVOKE reaches 68 % [48.41, 82.80] while every recovery-less baseline stays in 0 to 8 %, and even H2O (the strongest non-EVOKE comparator) manages only 8 % [2.22, 24.97]. On Qwen 3.6 35B-A3B (MoE + thinking, IQ2 quant) EVOKE drops to 52 % [33.50, 69.97] while every baseline including H2O is at 0 %; the absolute pass-rate falls with quantization aggressiveness but the relative advantage (5×+ over the best baseline) holds across all three architectures.

7.3 InfLLM validates the thesis: two schedulers on the same substrate

Section 7.2 established the structural divide: recovery-bearing policies pass, recovery-less baselines fail. The closest external-memory baseline (InfLLM, reimplemented on our substrate via `kv_block_load`) sits on our side of the divide. This subsection runs the head-to-head between the two schedulers that live on top of the shared memory-hierarchy substrate.

Both policies use identity-keyed recovery (the same `kv_block_load` primitive paste-back of the model's own K and V). Both treat eviction as a reversible cut. Both manage the cache as a hierarchy with an external-memory tier. They differ on *how* they schedule it. EVOKE evicts via a watermark policy with a multi-signal scorer (attention, recency, coherence, retrieval-embedding similarity) and

recovers top-$K{=}4$ per turn. InfLLM evicts more aggressively (to a sinks-plus-25 %-local-window footprint) and recovers a wider top-$K{=}8$ per turn via pure-retrieval scoring. The head-to-head is a comparison between two scheduler shapes on the same paging hardware.

The $n{=}5$ pilot in Section 7.2 shows the three recovery-bearing policies (EVOKE-with-K=4, EVOKE-with-attention, InfLLM-with-K=8) clustered between 48 and 64 percent with overlapping CIs. We extended the multifact sweep to 15 seeds at three budgets, 75 facts per cell, on both fully-swept architectures. The tighter intervals reveal a budget-regime crossover rather than one scheduler strictly dominating (Table 5).

strategy	b**=512**	b**=1024**	b**=2048**
Qwen 2.5 7B			
infllm	**81.3 % [71.1, 88.5]**	58.7 % [47.4, 69.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	50.7 % [39.6, 61.7]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	49.3 % [38.3, 60.5]	58.7 % [47.4, 69.1]	**65.3 % [54.1, 75.1]**
Llama 3.1 8B			
infllm	**81.3 % [71.1, 88.5]**	65.3 % [54.1, 75.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	58.7 % [47.4, 69.1]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	60.0 % [48.7, 70.3]	57.3 % [46.1, 67.9]	**66.7 % [55.4, 76.3]**
recency	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]
streaming_llm	0.0 % [0.0, 4.9]	1.3 % [0.2, 7.2]	13.3 % [7.4, 22.8]
h2o	0.0 % [0.0, 4.9]	8.0 % [3.7, 16.4]	29.3 % [20.2, 40.4]
snapkv	1.3 % [0.2, 7.2]	16.0 % [9.4, 25.9]	42.7 % [32.1, 53.9]

Table 5: Multi-fact pass rate at $n{=}15$ (75 facts per cell) with Wilson 95 % CIs. Recovery-less baselines (bottom block) shown once for the Llama sweep; the Qwen sweep produces the same cluster ($\leq$25 % at the loosest budget). At $b{=}512$ InfLLM statistically separates from EVOKE on both architectures (non-overlapping Wilson CIs). At $b{=}2048$ EVOKE leads with overlapping CIs.

EVOKE and InfLLM trade places along the budget axis. At $b{=}512$ InfLLM's larger $K{=}8$ retrieval catches more relevant blocks per turn and statistically separates from EVOKE on both architectures (Qwen non-overlapping CIs: [71.1, 88.5] vs evoke_kv_restore [39.6, 61.7]; Llama non-overlapping CIs: [71.1, 88.5] vs evoke_attention [48.7, 70.3]). At $b{=}1024$ the policies tie. At $b{=}2048$ EVOKE pulls ahead but with overlapping CIs, so we report this as directional rather than separated. Pure recovery-less baselines remain $\leq$25 % at the loosest budget with tight $n{=}15$ CIs.

Why InfLLM wins at tight budget. The per-fact failure cross-tab shows the "date" fact (a Treaty of Vrenholm probe) is structurally hard for every strategy ($\leq$1/5 across all nine strategies). EVOKE's $K{=}4$ selection misses 91 % of date cells while InfLLM's $K{=}8$ misses 67 %, indicating that the EVOKE-vs-InfLLM gap at tight budget is selection-failure-dominated, not substitution noise. Both policies use the same recompute-free recovery primitive; what InfLLM does better at tight budget is select a wider net of candidate blocks per recovery pass.

Why EVOKE wins at loose budget. At $b{=}2048$, headroom lets the attention-driven scorer keep more of the right blocks resident in the first place (the `evoke_attention` configuration is the top scorer on both architectures at the loosest budget). InfLLM's fixed aggressive footprint (sinks plus a 25 % local-window tail) does not adapt to budget headroom; EVOKE's watermark-driven eviction does. The Llama capture-layer ablation in Appendix A.9 reinforces this reading: changing the scorer's capture layer does not change the residency set on Llama at $b{=}1024$, because selection is what the workload bottlenecks on.

What the head-to-head means for the thesis. The crossover is a result *within* the thesis, not against it. Both schedulers are recovery-bearing; both clear the structural divide that separates them from recency, StreamingLLM, H2O, and SnapKV. The fact that InfLLM at $K{=}8$ beats EVOKE at $K{=}4$ on tight-budget multifact is a statement about per-turn recovery breadth, not about whether the cache should be paged virtual memory. Both schedulers answer that question the same way, and the data backs both. **The win the paper claims is the thesis the two schedulers share: identity-keyed recovery, eviction-as-reversible-cut, cache-as-hierarchy. The scheduler choice is second-order,**

regime-dependent, and reported honestly here so that practitioners pick the right K for their budget rather than read either scheduler as universally better.

7.4 Wall-clock head-to-head: parity with truncate, not advantage

We run the same 14-turn planted-fact session through three server policies via `scripts/baseline_bench.py`: `no_eviction` (high_watermark=0.999, budget lifted to `n_ctx`; what a vanilla OpenAI-compatible server with prefix caching does), `truncate` (StreamingLLM-style: drop the oldest non-sink block under budget pressure, evicted content gone), and `evoke` (default scoring with `kv_restore`). Qwen 2.5 7B, b=1024 where applicable, `n_ctx=16384`, n=15 runs, 95 % CI via t-distribution.

policy	budget	active end	evict	recov	probe	elapsed (95 % CI)
no_eviction	16384	2476	0	0	PASS	18.90s [18.74, 19.05]
truncate	1024	**685**	**70**	0	PASS	22.11s [19.46, 24.76]
evoke	1024	**684**	**90**	**24**	PASS	**21.20s [21.07, 21.33]**

Table 6: 14-turn planted-fact head-to-head on Qwen 2.5 7B, n=15. All 15 runs of every policy answer the probe correctly. EVOKE's [21.07, 21.33] interval lies inside truncate's [19.46, 24.76] interval; they are not statistically distinguishable at α=0.05.

The headline is recall and footprint parity with truncation, not a speed win. EVOKE and truncate both sit at 684 to 685 active tokens (roughly 3.6× smaller than no_eviction's 2476 footprint, inside the 1024-token budget), while no_eviction only fits because this particular 14-turn session is within `n_ctx`; a longer session crashes with no available KV slot. On wall-clock, EVOKE's [21.07, 21.33] interval lies inside truncate's [19.46, 24.76] interval (truncate's variance is dominated by intrinsic SSH-launch start-up jitter), so the two are not statistically distinguishable at α=0.05.

The win EVOKE claims here is not throughput. It is recovery: EVOKE matches no_eviction's recall at ∼3.6× smaller cache and at truncate-parity wall-clock while restoring the missing fact through 24 `kv_block_load` splices rather than per-turn re-prefill, a capability truncate fundamentally lacks. In this particular 14-turn session the planted fact happens to stay in truncate's recency window, so it passes here; the agent-bench scaffold in Section 7.2 runs the same fact past recency and truncate fails at every budget. The session-length scaling sweep in Appendix A.7 extends the comparison to $T \in \{28, 56\}$ turns and confirms the wall-clock gap remains within ∼10 % across the 4× scaling.

The no_eviction elapsed time is the natural latency floor for a stateful server: every turn's prefix matches the previous cache byte-for-byte and only the new tail decodes. It is what a paged-attention or prefix-caching server can hit when the cache has room; EVOKE's overhead over this floor (2.30 seconds, ∼12 %) is the cost of pretending the budget is 1024 instead of 16,384.

7.5 Substrate portability: what does not transfer to vLLM

The headline finding is negative. We ported both the EVOKE policy layer and the recompute-free recovery primitive to vLLM v1 with PagedAttention (Kwon et al., 2023) on a fork (`github.com/Anyesh/vllm`). The primitive composes from existing kernels with no CUDA-side work; the policy layer's similarity-based smart-recovery does not land on vLLM V1 because the V1 scheduler has no session-scoped logical position space for the recovered bytes to occupy. The port surfaces a missing abstraction on production paged substrates; it does not produce a working similarity-recovery system on vLLM.

What composes (the primitive). The recovery primitive (paged byte transfer plus RoPE re-anchoring) composes from vLLM's existing `swap_blocks_batch` and `rotary_embedding` kernels without modification. The per-block RoPE re-anchoring is `ops.rotary_embedding(inverse=True)` plus a forward call; on this substrate we did not have to add a kernel, only a Python rotator that arranges the existing ops in the right order on the transfer stream. The rotation reproduces direct positional rotation to within 1.6×10^{-2} in bf16 max absolute deviation across a synthetic K-vector benchmark (the claim is that the inverse+forward composition is a numerical no-op up to expected storage-dtype round-off). The `EvokeCachePolicy` registers as

a first-class choice in vLLM's offload-tier `_CACHE_POLICIES` registry alongside LRU and ARC and is invoked on every offload-tier eviction decision.

What does not compose (the policy layer). The policy layer has a precondition that vLLM v1 does not satisfy. The V1 scheduler treats cached tokens as a contiguous prefix from position 0 of the active request's prompt (`num_computed_tokens` $\leq$ `request.num_tokens`); it has no session abstraction across requests, no logical position space that outlives a single `generate()` call. Similarity-based smart-recovery (the recall of a block whose original position is in some past sequence and whose content is semantically relevant to the new request's query) we did not find a way to land in that model that preserves the V1 scheduler's invariants: the scheduler has no logical-position slot for the recovered bytes outside the active request's prompt range. llama.cpp provides this slot natively (a session is a single sequence whose logical positions accumulate across turns); vLLM V1 does not.

The mechanism that *does* transfer to vLLM is same-content gap-fill: when a multi-turn agent re-sends growing history each turn, the prompt's prefix-cache hashes match previously offloaded blocks, and the eviction policy on the offload tier governs which prior-turn blocks were retained under capacity pressure. We ran a one-cell NIAH-style probe on this regime (`scripts/niah_vllm_samecontent.py` in the fork): five distinct needle prompts on Qwen 2.5 7B Q4_K_M, then 30 unrelated filler prompts ($\sim$51 k filler tokens into a 25 k-token GPU KV cache with a 256 MiB CPU offload tier), then re-sends each needle prompt with a probe question. The two runs differ only in `kv_connector_extra_config["eviction_policy"]` (`"evoke"` vs `"lru"`). Both policies score 5/5 probe-OK; per-probe wall-clock medians are 2.06 s for EVOKE and 1.80 s for LRU across the five probes (n=5 per arm, no seed sweep, no CIs reported). The needle text is in the re-sent prompt itself, so correctness on this metric does not distinguish the policies. Under vLLM's request-scoped model, the EVOKE policy's residual lever is eviction ordering in the offload tier, and on a probe whose correctness is decided by the prompt rather than the cache that lever does not move the metric.

The finding names a concrete precondition for the policy layer (a session-scoped logical position space) and a concrete substrate (vLLM v1) where that precondition is absent. The follow-up scoped in Section 9 is correspondingly a host-side session abstraction rather than further port engineering.

8 Discussion and Limitations

Hybrid memory plus thinking-mode interaction. On hybrid memory models, Qwen 3.5's `<think>...</think>` trace is by default stripped from the API response but kept in the physical KV cache. Strict alignment would require evicting the thinking range from the attention cache, but `llama_memory_hybrid::seq_rm` rejects partial tail rollback of the recurrent half (a Mamba state cannot be partially sliced) and aborts. EVOKE's eviction respects the rejection and the session resets on the divergent turn (one such reset is observed in the Qwen 3.5 demo at filler 9). The workaround that ships today is `EVOKE_SUPPRESS_THINKING_STRIP=1`: the server keeps the thinking trace in returned content, the client echoes it back, and the cached state stays aligned. A future no-shift eviction mode (using the attention-only `llama_kv_block_seq_rm`/`seq_add` primitives already in the fork) would let the thinking range be evicted from attention while leaving the recurrent state untouched.

Recurrent and pure-SSM models. EVOKE operates on the attention component of hybrid models. Pure-SSM models such as Mamba and RWKV are out of scope, since they have no traditional KV cache. The blurry memory of an SSM is preserved naturally on a hybrid model, because EVOKE does not touch it.

Memory accounting and deployment realism. Saved blocks live in host RAM, and the cost is real. For Qwen 2.5 7B at `block_size=128`, one cell costs 56 KiB (28 layers $\times$ 2 (K and V) $\times$ 512 `n_embd_kv_gqa` $\times$ 2 bytes), one block is 7 MiB, and a 1000-block session costs roughly 7 GiB of host RAM. A multi-tenant deployment co-locating N sessions pays $N \times 7$ GiB at this footprint. On Llama 3.1 8B with the larger per-token K/V footprint ($\sim$131 KB/token vs Qwen's $\sim$60), the figure scales correspondingly. The host RAM tier is therefore a deployment-cost line item, not free, and at a 7B-class model on a single-tenant box ($\sim$32–64 GiB host RAM typical) it is binding at $\sim$5–9 concurrent sessions before `kv_restore_ram_budget_bytes` starts demoting to the disk-spill tier

(Appendix A.5 measures graceful degradation under exactly this pressure). Both LRU bounding (`kv_restore_ram_budget_bytes`) and the disk-spill tier (`kv_restore_spill_path`) are off by default; operators running multi-tenant should turn at least the LRU bound on. The trade EVOKE makes is host RAM proportional to evicted content in exchange for exact recovery; compression schemes that summarize evicted KV (compressive transformers (Rae et al., 2019)) trade the reverse direction.

Multi-session and the model-in-VRAM bottleneck. The session pool in Section 5 swaps KV state, but the model weights remain a single instance in VRAM. With a 7B Q4_K_M model consuming roughly 5 GB of VRAM, a 16 GB GPU has roughly 11 GB left for active KV, swap-in temporary state, and Flash Attention working memory. Concurrency is therefore bounded by per-session KV footprint, not by EVOKE's policy layer. Production multi-tenant deployments would need either tensor-parallel weight sharing or thinner per-session quantization to lift this ceiling.

Smart-recovery is a small retrieval system, by design. Policy-layer quality is bounded by the retrieval encoder's discrimination on the workload: a poor encoder picks the wrong blocks to recover, even though the bytes spliced back into the cache are still the model's own K and V. The architectural line between identity-keyed substitution and similarity-shaped selection is drawn once in Section 4.

The contribution boundary is the recovery primitive, not the eviction scorer. The $n{=}15$ multi-fact sweep (Section 7.3) and its Llama 3.1 8B replication (Appendix A.9) show that the EVOKE-vs-InfLLM ranking flips along the budget axis. At tight budgets the win goes to whichever policy makes fewer selection failures, and a larger-K pure-retrieval recovery (the InfLLM adaptation at $K{=}8$) catches more blocks per turn than EVOKE's $K{=}4$ attention-driven selection; the per-fact failure cross-tab in Section 7.3 confirms the gap is selection-failure-dominated, not substitution noise. At loose budgets headroom lets a smarter scorer pay off and the attention-driven EVOKE configuration takes the top spot. The honest framing of the contribution is therefore: the recompute-free identity-keyed K/V splice is what divides recovery-bearing policies from recovery-less ones (the 20–40 % to 76–100 % gap on NIAH, the 0 % to 50–80 % gap on multifact), and the attention scorer is a regime-targeted improvement on top of the primitive rather than the load-bearing claim. The Llama capture-layer ablation in Appendix A.9 reinforces this: changing the scorer's capture layer does not change the residency set on Llama at budget 1024, because selection is what the workload bottlenecks on.

Cross-architecture latency depth. Sections 7.1, 7.2 and 7.4 and appendices A.1, A.6 and A.7 measure latency and scorer ablations on Qwen 2.5 7B; Appendix A.9 adds an end-to-end Llama 3.1 8B run covering NIAH, multifact, and agent_bench. The per-token K/V byte footprint scales predictably with model size (layers $\times$ heads $\times$ head dim $\times$ 2 $\times$ element width), so the host-to-GPU memcpy cost that dominates `kv_block_load` at long blocks shifts in a known direction with parameter count, GQA grouping, and quantization width. We measured two architectures end-to-end; the cost model predicts where the band moves for others but the actual band on those substrates is not measured in this paper.

Embedding quality. Smart-recovery scoring uses `bge-small-en-v1.5` (384-dim), sufficient for the NIAH workload in Section 7.2. The embedder is swappable via `EvokeConfig.use_retrieval_embeddings`; larger or task-tuned encoders are an obvious sweep we have not run.

Smart-recovery placement. Recovery currently appends blocks at the cache tail before the new user message is decoded, so recovered blocks land as earlier context the model attends back to from the freshest probe. This fits the chat-completions pattern. For streaming agent workloads where the model is generating tokens that subsequently need earlier context, an in-place insertion path (using the attention-only `llama_kv_block_seq_rm`/`seq_add` primitives) would let recovered blocks slot back at their original logical position.

9 Future Work

- **Session abstraction for paged substrates.** Section 7.5 measures the EVOKE port against vLLM v1 and identifies a missing ingredient on that substrate: a logical position space that outlives a single request. The next-paper question is whether a host-side session abstraction (e.g. a `session_id` on `Request` with cache scoped to it) can be added to vLLM without breaking the V1 scheduler's contiguous-prefix invariants, and how that abstraction would compose with SGLang's RadixAttention (Zheng et al., 2024) and LMCache (LMCache Team, 2024). The transfer mechanism we need (paged scatter/gather) already exists in vLLM via `swap_blocks_batch`; the open work is on the scheduler-side abstraction that gives the transfers a session-scoped destination.
- **iSWA end-to-end on Gemma.** The fork-side `llama_kv_block_save` and `llama_kv_block_load` now handle both base and SWA sub-caches via `llama_get_kv_cache_swa()` (fork commit `1f37e75e7`). The remaining engine-side bookkeeping for the dual-cache surface and a Gemma 3 or 4 GGUF end-to-end smoke through the existing demo and agentic-eval scripts is the next step.
- **No-shift eviction mode for hybrid plus thinking.** Use the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives (already in the fork) to evict attention cells without compacting, so a hybrid model's recurrent state stays in sync while the thinking range is dropped from attention.
- **Multi-layer scorer.** The fork already exposes `llama_attn_capture_set_layers` for up to 16 layers per decode. The current scorer uses a single mid-layer; a more principled scorer could average across an early, a middle, and a late layer to pick up both syntactic and semantic relevance signals.
- **Tensor-parallel and quantized variants.** Validate that the C primitives work unchanged across multi-GPU tensor-parallel layouts and across the full Q3 to Q8 quantization range.
- **Continuous learning of harness priorities.** Today `evoke_priority` is set by the harness. A reasonable next step is for EVOKE to learn priority from observed attention patterns, so a harness that does not set priorities still gets the benefit.
- **Multi-fact, judge-scored, randomized-position bench.** Section 7.3 already exercises five facts per session with a model judge; lift this to a RULER- or LongBench-style suite with per-position jitter and partial-credit scoring so the metric is not a binary single-probe hit.
- **Real-agent traces.** The benches in this paper are synthetic (NIAH, planted-fact multifact, a single-probe agent harness). The natural next step is a head-to-head on a real workload such as SWE-bench Verified, τ-bench, or a recorded Claude Code session, where the cache pressure, file-read distribution, and probe shape are not constructed by us. EVOKE's stateful server is already wire-compatible with the OpenAI chat-completions API used by those harnesses, so the engineering work is the eval setup rather than the system port; we name this as future work and decline to extrapolate the synthetic results onto it.
- **KV-quantization head-to-head against KIVI.** Appendix A.8 measures Q4_0 only. KIVI's per-channel-per-token asymmetric scheme is specifically designed to keep generation viable at low bit-widths and is the experiment that would settle the "why not just quantize?" question for the production-quality quantization regime.

10 Conclusion

EVOKE treats the KV cache as a small fast tier of a memory hierarchy. Blocks the model is not currently attending to leave the cache to host RAM at metadata cost, and when a future turn needs them they are spliced back as their original K and V tensors with one RoPE phase shift. On Qwen 2.5 7B, the recovery primitive runs in 0.5 to 7 milliseconds across block sizes from 20 to 1280 tokens, 20 to 32 times faster than re-prefilling the same tokens through the model (5.9 to 7.5$\times$ over the full save+load lifecycle, factoring in the eviction-side save cost); on Llama 3.1 8B the band is 1.78–15.66 ms and 7–15$\times$ (2.6–2.8$\times$ lifecycle). On a 14-turn planted-fact session (n=15), it matches the unconstrained `no_eviction` baseline's recall at $\sim$3.6$\times$ smaller cache footprint and at `truncate`-comparable wall-clock (21.20s [21.07, 21.33] vs 22.11s [19.46, 24.76]; `truncate`'s SSH-launch jitter dominates its CI so the two are not statistically distinguishable at α=0.05). Across the agent_bench and multi-fact

head-to-heads, the recovery-bearing policies (`evoke_kv_restore`, `evoke_attention`, and the same-substrate `infllm` adaptation) recover at every budget; every recovery-less baseline (recency, StreamingLLM, H2O, SnapKV) either fails outright (agent_bench, all budgets) or trails the recovery-bearing cluster by a wide margin (multi-fact n=15: recovery-less $\leq$43 %, recovery-bearing $\geq$56 %); the same divide holds on single-needle NIAH, with SnapKV's heavy-hitter retention on Llama at looser budgets (Appendix A.9) as the only documented exception. The distinction at depth is presence of a recovery primitive, not the smartness of the eviction scoring. The eviction-scoring choice is regime-dependent: at tight budgets, where selection failures dominate, a larger-K pure-retrieval recovery (the InfLLM adaptation) is competitive with or beats the attention-driven scorer; at loose budgets, where headroom lets a smarter scorer pay off, the attention-driven EVOKE configuration takes the top spot (Section 7.3 and appendix A.9). The load-bearing contribution is therefore the recompute-free recovery primitive itself, with the attention scorer as a regime-targeted improvement on top of it.

Code and Reproducibility

The EVOKE policy layer (Python) is at `github.com/Anyesh/EVOKE` under Apache 2.0. The forked llama.cpp with the C primitives is at `github.com/Anyesh/llama.cpp` under MIT (the upstream license). The partial vLLM port referenced in Section 9 is at `github.com/Anyesh/vllm` under Apache 2.0 (the upstream vLLM license). Every number in Section 7 was produced by a script in `scripts/` checked into the EVOKE repository, with the exact invocations listed in Appendix B. Raw output files for the agentic eval and the keepalive eval are checked in under `results/`.

References

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.

Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv preprint arXiv:1911.05507*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing 568:127063, 2024*.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv preprint arXiv:2402.04617*.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*.

Feng, Y., Lv, J., Cao, Y., Xie, X., & Zhou, S. K. (2024). Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv preprint arXiv:2407.11550*.

Cheng, J., Liu, Y., Wang, K., Xiang, X., Yu, X., Liu, J., Yi, F., & Jiang, J. (2024). LMCache: 7x Throughput Improvement Through Layered, Cross-Engine KV Cache. *vLLM Production Stack project*.

Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., & Lin, W. (2024). Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv preprint arXiv:2401.02669*.

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-Pack: Packed Resources For General Chinese Embeddings. *SIGIR 2024*.

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM 2024*.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL 2024*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.

Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024*.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., & Han, S. (2024). Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML 2024*.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., & Han, S. (2024). DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv preprint arXiv:2410.10819*.

Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., & Panwar, A. (2024). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437*.

Qwen Team. (2024). Qwen 2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

A Detailed Evaluations

This appendix collects evaluations that are referenced from the main body but whose full detail would crowd the headline results.

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	h2o	fail	34	0	0.00
512	snapkv	fail	34	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.25
512	evoke_kv_restore	PASS	35	1	**3.13**
512	evoke_attention	PASS	31	0	**0.01**
512	infllm	PASS	39	1	3.12
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	h2o	fail	24	0	0.00
1024	snapkv	fail	24	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.40
1024	evoke_kv_restore	PASS	28	1	**3.86**
1024	evoke_attention	PASS	26	1	3.75
1024	infllm	PASS	35	1	3.76
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	h2o	fail	4	0	0.00
2048	snapkv	fail	4	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.70
2048	evoke_kv_restore	PASS	10	1	**3.89**
2048	evoke_attention	PASS	8	0	**0.00**
2048	infllm	PASS	31	1	3.07

A.1 Per-cell agent_bench (scorer ablation context)

The full per-cell agent_bench table (referenced in Sections 7.2 and 7.4), with eviction counts and per-recovery latency, on Qwen 2.5 7B at `block_size=64`:

The `evoke_attention` row at b=512 is the differential against the heuristic scorer at the tightest budget: with attention scoring, the planted fact block is never evicted in the first place (the model's attention across the unrelated filler turns keeps landing on the config-file block, so the scorer assigns it a high score and eviction picks lower-attended document blocks instead). Total evictions drop by one (35 to 34) and recoveries go to zero, and the probe still passes. At 1024 and 2048 the two strategies converge: recency is sufficient when the budget can hold most history naturally.

A.2 Server eviction demo

A 14-turn session driven directly against the chat-completions API. The script plants a fact at turn 1 ("favorite number = 4242"), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The recorded demos are kept as visual proof of the per-turn evolution of `active_tokens` against the budget.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the `KVRestoreBackend` RAM budget configured tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls "4242" at the probe.

Qwen 3.5 9B (hybrid Mamba+Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn. The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

A.3 Default-knob ablations

We sweep one policy knob at a time across the full 25-cell NIAH grid (5 needles $\times$ 5 depths) at b=1024 on Qwen 2.5 7B, holding everything else at the recommended `evoke_attention` configuration:

knob	swept values	pass rate per value
`block_size`	{32, 64, 128, 256, 512}	84, **100**, 80, 56, 20 %
`attention_capture_layer`	{4, 10, 14, 20, 24, 27}	all 100 %
`smart_recover_k`	{1, 2, 4, 8, 16}	all 100 %
`w_coherence`	{0.0, 0.2, 0.4, 0.6, 0.8}	all 100 %
`recent_tail_protect_frac`	{0.0, 0.05, 0.1, 0.2, 0.3}	all 100 %

The `block_size` curve is the load-bearing ablation: `block_size=64` hits the headline 100 % pass rate, and degradation is symmetric on both sides. Finer-grained blocks (`32`) fragment fact-bearing content across two neighbouring blocks; coarser blocks (`128, 256, 512`) dilute the block embedding's representative signal with adjacent haystack content. The other four knobs are flat at 100 % on this workload; tighter-budget or multi-fact regimes (Section 7.3) where eviction pressure is sustained throughout the session would likely surface knob-level differentiation.

A.4 Smart-recovery policy factorial

The smart-recovery loop in Section 5 has three named policy decisions: (K1) score using a dedicated retrieval-tuned embedder applied to the raw user-message text, versus pooling LM hidden states; (K2) run the recovery splice before the new user-message tail is decoded, versus after; (K3) gate the top-k recovery against the strongest non-current-turn resident block's similarity, versus skipping. We run the full 2^3 factorial on NIAH at b=512 on Qwen 2.5 7B (25 cells per row).

cell	K1	K2	K3	pass	Wilson 95 % CI
000	off	off	off	28.0 %	[14.3, 47.6]
001	off	off	on	28.0 %	[14.3, 47.6]
010	off	on	off	0.0 %	[0.0, 13.3]
011	off	on	on	0.0 %	[0.0, 13.3]
100	on	off	off	76.0 %	[56.6, 88.5]
101	on	off	on	76.0 %	[56.6, 88.5]
110	on	on	off	96.0 %	[80.5, 99.3]
111	on	on	on	96.0 %	[80.5, 99.3]

Marginals over n=100: K1 (raw-text retrieval embedding) +72 pp (14 % off vs 86 % on); K2 (recover-before-decode) -4 pp main effect but +20 pp conditional on K1 on, -28 pp conditional on K1 off; K3 (resident-gate) 0 pp on this benchmark. K1 is the dominant policy decision by a wide margin: the bge-small retrieval embedder widens the cosine band from 0.85 to 0.93 (LM hidden states) to 0.4 to 0.9, and the top-k selection becomes discriminative enough to actually pick the needle block over haystack noise. K2's asymmetry has a mechanical explanation: recovering before decode splices the recovered block earlier in the cache than the probe, so the model attends back to it; this is a win when retrieval is good (K1 on) and a regression when retrieval is bad (K1 off, the wrong block lands as earlier context). K3 contributes no measurable effect on this benchmark at b=512. The resident-gate was added to address a depth-95 % failure mode where a top-k recovery brought back a weaker match than the already-resident needle, but this factorial's metric does not expose that mechanism. At the budget and benchmark we ran the factorial against, K3 is not load-bearing.

A.5 Host-RAM pressure: graceful degradation

The four-tier saved-block pool (RAM-resident $\rightarrow$ disk-spilled $\rightarrow$ breadcrumb-only $\rightarrow$ discarded) is exercised under tight host memory by `scripts/hostram_bench.py`: a 20-fact session against an 80-paragraph haystack at b=1024 with `kv_restore_ram_budget_bytes` set to hold only 8 saved blocks ($\sim$29 MiB of K/V on Qwen 2.5 7B), with the disk-spill tier enabled.

Outcome: across the session, 207 blocks are evicted from active KV. 200 of them spill to disk, 4 stay RAM-resident at the end (the LRU pool), and 3 fall through to breadcrumb-only. No block is

discarded outright; `lru_drops` = 0 because the spill tier catches every saved-pool eviction. Smart-recovery successfully restores 4 of 20 facts under this ∼14 % RAM ceiling, well below the ∼60 % rate the same workload would hit with unbounded RAM, but the system serves the entire session without crashing and the recovery pipeline reaches the correct stored bytes when it does pick them. The spill tier acts as the slow-path safety net rather than the dominant tier in production deployments configured with a larger RAM budget.

A.6 Persistent-reference (keepalive) ablation

The Appendix A.1 differential is observable only at the tightest budget because the filler turns in agent_bench are semantically unrelated to the planted fact: the model's attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. We repeat the agentic eval with each of the ten filler-file contents referencing "the retry policy from config.py", and the final probe phrased "looking at the retry policy in config.py, what is the maximum retry limit?", a question whose answering tokens have strong attention to the early config block.

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	**PASS**	**21**
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	**PASS**	**11**
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

This bench is a scorer-only ablation: the harness does not invoke recovery. A block the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the two budgets where the heuristic path fails outright. Attention-based scoring sees the model's own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it. The heuristic-only scorer evicts the config block under pressure and the fact is gone before the probe runs. Recompute-free recovery is a strong second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place.

A.7 Session-length scaling and the recovery-aware fix

The Section 7.4 head-to-head is fixed at 14 turns. The scaling sweep runs the same workload at $T \in \{14, 28, 56\}$ turns, 5 seeds per cell, and produces a paired curve: a baseline curve with the production scorer, and a recovery-aware curve with `w_recovery=1.0`.

turns	policy	active	evict	recov	elapsed (s, 95 % CI)
14	no_eviction	2476	0	0	19.19 [18.76, 19.63]
14	truncate	685	66	0	21.33 [20.76, 21.90]
14	evoke (baseline)	684	90	24	21.58 [21.17, 21.99]
14	evoke (recovery-aware)	683	65	4	21.79 [21.54, 22.04]
28	no_eviction	6221	0	0	51.05 [50.10, 52.00]
28	truncate	717	486	0	65.14 [64.35, 65.94]
28	evoke (baseline)	721	566	80	68.06 [67.23, 68.89]
28	evoke (recovery-aware)	616	507	20	67.84 [66.65, 69.03]
56	no_eviction	12607	0	0	106.11 [105.19, 107.03]
56	truncate	674	2316	0	170.41 [169.27, 171.54]
56	evoke (baseline)	664	2522	192	182.36 [180.67, 184.06]
56	evoke (recovery-aware)	675	2400	20	185.73 [184.98, 186.48]

EVOKE (either curve) holds the active-token footprint at ∼670 to 720 tokens across the 4× session-length sweep, while `no_eviction` grows linearly to 12 607 tokens at T=56. Wall-clock is comparable to truncate within roughly 7 to 10 % across the sweep, not faster. We do not claim a

speed win at long sessions; the recovery capability has a real cost and truncate has nothing to recover. The honest pitch is identical memory footprint plus the recovery capability truncate lacks.

The recovery-aware fix (`w_recovery` = 1.0) shuts down a recover-then-immediately-evict thrash: recoveries drop from 24/80/192 to 4/20/20 across $T{=}14/28/56$ (-83 to -90 %), and evictions drop in lockstep. Wall-clock does not improve because the retrieval embedder ($\sim$10 ms per query) substitutes for the saved thrash work at these session lengths. The fix is transparent on accuracy: multifact verification shows recovery-aware 48/56/64 % vs kv_restore 52/60/64 % at budgets 512/1024/2048, CIs heavily overlap. The mechanism is cleaner; the wall-clock parity is structural.

A.8 KV cache quantization on the measured substrate

A reviewer asked whether KV-cache quantization is the cheaper alternative to eviction. We measured llama.cpp's stock `GGML_TYPE_Q4_0` (per-tensor symmetric 4-bit), which is the production-available default in the runtime EVOKE targets. We did not run KIVI (Liu et al., 2024) or other per-channel-per-token asymmetric schemes; the result below speaks only to the Q4_0 substrate.

The Q4_0 result is uniform: NIAH, multifact, and agent_bench all collapse to 0 % at every budget. Representative agent_bench generation at $b{=}2048$ with full 5962-token context preserved at Q4 precision: `‘’, and, and, and, and, and, and, and, and...’’`. F16 EVOKE at $b{=}1024$ retains 96 to 100 % NIAH on the same workload. On this substrate (Q4_0, Qwen 2.5 7B) 4-bit KV quantization is not a usable alternative to eviction. We do not extrapolate this to KIVI or other per-channel asymmetric schemes, whose per-channel structure is specifically designed to keep generation viable at lower bit-widths; running them is the next experiment for a deployer choosing between policy-layer eviction and aggressive KV quant.

A.9 Llama 3.1 8B: cross-architecture detail

NIAH and multifact pass-rates on Llama 3.1 8B are included in the main tables (Tables 3 and 5). A capture-layer ablation on the same NIAH grid (`EVOKE_ABL_DIM=attention_layer`, values $\{8, 16, 20, 24, 28\}$) yields identical pass-rates at both regimes: 88 % [70.04, 95.83] (22/25 cells) across all five capture depths at $b{=}1024$, and 76 % [56.57, 88.50] (19/25 cells) across all five capture depths at $b{=}512$, with the same cells failing at every layer in each budget. Neither the 12-pp Qwen-vs-Llama gap at $b{=}1024$ nor the 20-pp gap at $b{=}512$ is a capture-layer tuning miss; both reflect architectural differences in Llama's attention patterns or selection-step bottlenecks. The retrieval encoder picks the same residency set regardless of which capture layer feeds the eviction scorer. This validates the "ports without modification" claim: capture-layer choice is not load-bearing on Llama at any budget we tested.

The three EVOKE variants (`kv_restore`, `attention`, `recovery_aware`) converge to identical NIAH pass rates (76/88/88 %) confirming the recovery-aware fix is transparent on single-needle retrieval. Agent_bench on Llama 3.1 8B: EVOKE strategies (breadcrumb, kv_restore, attention) and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail (the same divide as Qwen).

A.10 Concurrency and PCIe bandwidth

We drive $N \in \{1, 4, 8, 16\}$ concurrent sessions through the same evoke_serve instance (Qwen 2.5 7B, $b{=}1024$, smart-recovery on, 14-turn planted-fact workload per session). Each level repeats for five rounds so the per-turn latency distribution at $N{=}16$ is built from 1120 turn samples.

N	probe_ok	n_turn	wall (s)	p50	p95	p99	p999	max
1	5/5	70	38.9	2.71	3.04	3.10	3.10	3.19
4	20/20	280	51.5	3.56	4.52	4.87	4.87	4.90
8	40/40	560	98.8	6.89	8.95	9.10	9.14	9.16
16	80/80	1120	205.0	14.86	17.77	18.01	18.13	18.14

Table 7: Per-turn latency in seconds at varying concurrency. Probe pass is 100 % at every N; per-round wall-clock at $N{=}16$ varies by under 0.3 % across the five rounds.

Two findings the single-shot runs did not surface. The per-turn latency distribution is essentially deterministic under sustained load: at every N, the p999 latency is within 0.13 s of the p99 latency. There is no long tail; the latency band is set by the GPU's per-decode compute time. The 100 % probe-OK rate holds across all 80 $N{=}16$ sessions, so the K/V splice primitive holds up to 16 concurrent evicting sessions without correctness loss.

Per-session wall-clock scales roughly linearly with N ($\sim$23$\times$ at 16$\times$ concurrency). The dominant scaling factor is GPU compute contention (a single 16 GB GPU runs one Qwen 2.5 7B forward pass at a time), not the PCIe save/load path. Back-of-envelope: at 56 KiB per token times 128-token blocks ($\sim$7 MiB per block), N simultaneous eviction events of k blocks each move $7Nk$ MiB across PCIe Gen4 16 GB/s, $\sim$0.4 ms per block per session; at $N{=}16$ and a 4-block eviction burst the aggregate PCIe traffic is 448 MiB at $\sim$28 ms serial worst case. The PCIe pessimism is not the binding constraint here; per-GPU compute throughput is. Production multi-tenant scaling (the next-paper concern, Section 9) would need tensor-parallel or paged-virtual KV layout to lift the compute ceiling, which this paper does not claim to provide.

B Reproducing the Numbers

All numbers in Section 7 were produced on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Python 3.12) against Qwen 2.5 7B Instruct (`Qwen2.5-7B-Instruct-Q4_K_M.gguf` from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

B.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/libllama.so
```

B.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/libllama.so
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

B.3 Section 7.1 latency table (`profile_recover.py`)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 7.1 are the means.

B.4 Appendix A.2 eviction demo (`eviction_demo.py`)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py
```

```
# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

B.5 Section 7.4 head-to-head baselines (`baseline_bench.py`)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/libllama.so/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
  uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy launch wrapper is in `scripts/` (commit `2b3cbea`).

B.6 Section 7.2 and appendix A.1 agentic eval (`agent_bench.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Appendix A.1 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

B.7 Appendix A.6 keepalive workload (`agent_bench_keepalive.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

B.8 Multi-session pool sanity check (`verify_multi_session.py`)

```
# Server must already be running (see A.4)
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

C The Fork

The llama.cpp fork is hosted at `github.com/Anyesh/llama.cpp`. The fork tracks upstream `ggml-org/llama.cpp` and adds the EVOKE-specific commits listed below.

C.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
`1f37e75e7`	EVOKE: iSWA dual-cache support in `kv_block_save/load` (#41)	`llama_get_kv_cache_swa()` and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
`151f1cf93`	EVOKE: multi-layer attention capture (up to 16 layers per decode)	`llama_attn_capture_set_layers(ctx, layers, n)` writes one slab per layer in a single decode
`713f202e8`	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: `llama_attn_capture_set_layer`, `_set_buffer`, `_get_dims`, `_get_written`
`9b6232446`	EVOKE: attention-only `seq_rm` and `seq_add` primitives	`llama_kv_block_seq_rm` and `llama_kv_block_seq_add` reach the attention sub-cache directly via `llama_get_kv_cache`, for "no-shift" eviction modes
`a29599f4c`	EVOKE: `kv_block` primitives now cover hybrid memory and mrope models	Extends `llama_get_kv_cache` to unwrap `llama_memory_hybrid` via `get_mem_attn()` and `llama_memory_hybrid_iswa` via `get_mem_attn()->get_base()`. Removes the mrope `seq_add()` and `get_can_shift()` guards.

C.2 The two recovery primitives, verbatim

From `include/llama.h`:

```
LLAMA_API size_t llama_kv_block_save(
    struct llama_context * ctx,
            llama_seq_id   seq_id,
               llama_pos   p0,
               llama_pos   p1,
                 uint8_t * dst,
                  size_t   cap);

// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.
LLAMA_API bool llama_kv_block_load(
    struct llama_context * ctx,
             llama_seq_id   seq_id,
           const uint8_t * src,
                  size_t   size,
               llama_pos   new_p0);
```

`save` returns the number of bytes written (or zero on error, e.g. `cap` too small). `load` returns true on success. The caller is responsible for the buffer allocation in both directions.

C.3 Build instructions

The fork builds cleanly against the upstream llama.cpp toolchain. Requirements: CMake $\geq$ 3.14, a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (we tested with 13.1).

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python` 0.3.x compatibility so a single loaded module backs both the Python wrapper and the EVOKE ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.

C.4 Attention-capture C API surface, verbatim

The full C API for attention-weight capture, summarized in Section 3.3, added to `include/llama.h`:

```
LLAMA_API void   llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void   llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void   llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void   llama_attn_capture_get_dims  (const llama_context *, int32_t *
    n_layers,
                                               int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```